Dust: 역전파 없이 Transformer 사전학습하기, 분석 리포트

게시용 버전 · 원문 https://qlabs.sh/research/dust · Q Labs Research, 2026-10 (Samip Dahal, Bishwas Mandal, Serdar Gülbahar, Akshay Vegesna) · 웹 논문 https://qlabs.sh/research/dust (arXiv 판 없음) · 코드 MIT (커밋 b20f7c0) · 분석일 2026-10-29 · 논문 그림 © 저자, 라이선스 표기 없음

요약

1. **무엇을 했나:** 선형층 출력(활성값)에 토큰마다 독립인 가우스 잡음을 더하고 토큰별 손실 변화로 보상해서, forward만으로 출력 오차를 추정하고 입력과의 외적으로 가중치 gradient를 만든다. 토큰 하나가 population 구성원 하나가 되는 "가상 population"이다.
2. **숫자는 맞다:** 본문 수치 12개, Table 1의 power law 극한 4개, 파라미터 수(37.75M), 공식 코드의 draw 배분이 부록과 모두 일치한다.
3. **"backprop을 넘는다"는 학습 극초기, 같은 토큰 기준이다:** 넘는 두 예산은 update 6번(100k)과 61번(1M)이다. 16k population의 update 한 번은 forward 약 13,600회, backprop step의 약 4,500배 연산이다.
4. **20M 극한 4.431 < backprop 4.633은 불안정하다:** 다섯 칸 중 한 칸을 빼고 다시 적합하면 2.57-4.68이 나오고, 64 칸을 빼면 backprop보다 높다. 맨 위 칸만 따로 튜닝되었다.
5. **c_max < 1은 "자기 토큰만 보상"하는 구조적 편향이다:** cosine 법칙 cos(K) = c_max/√(1 + c/K)를 유도하면 c = (d_out + 1) × 토큰별 gradient 비정합도다. numpy toy에서 c_max는 국소 gradient와 전체 gradient의 cosine과 일치했다. 학습 population(16k)의 cosine은 head를 빼면 0.16-0.57이다.
6. **확실한 것:** 같은 forward 수에서 가중치 공간 ES(EGGROLL)보다 훨씬 효율적이라는 결과와, population에 따른 cosine 상승 법칙은 견고하다.
7. **재현:** 하지 않았다(공식 코드는 GPU 전용). 표 재계산, 연산량 환산, numpy toy만 CPU로 했다.

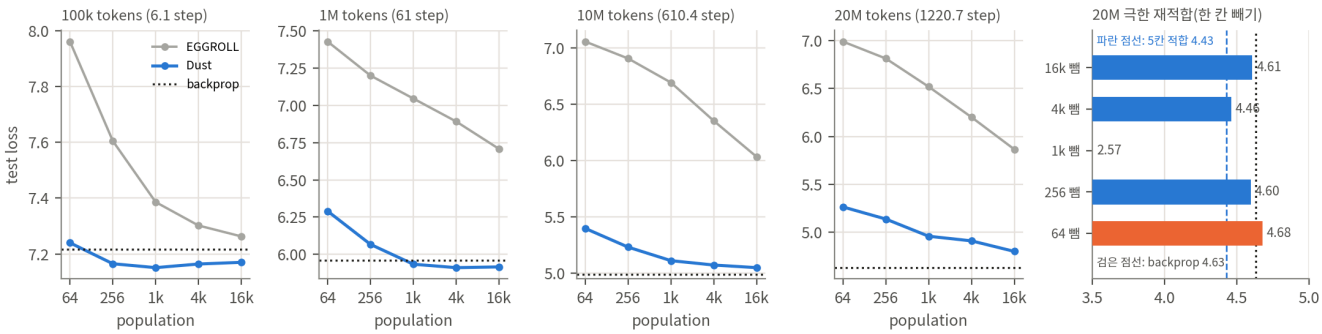
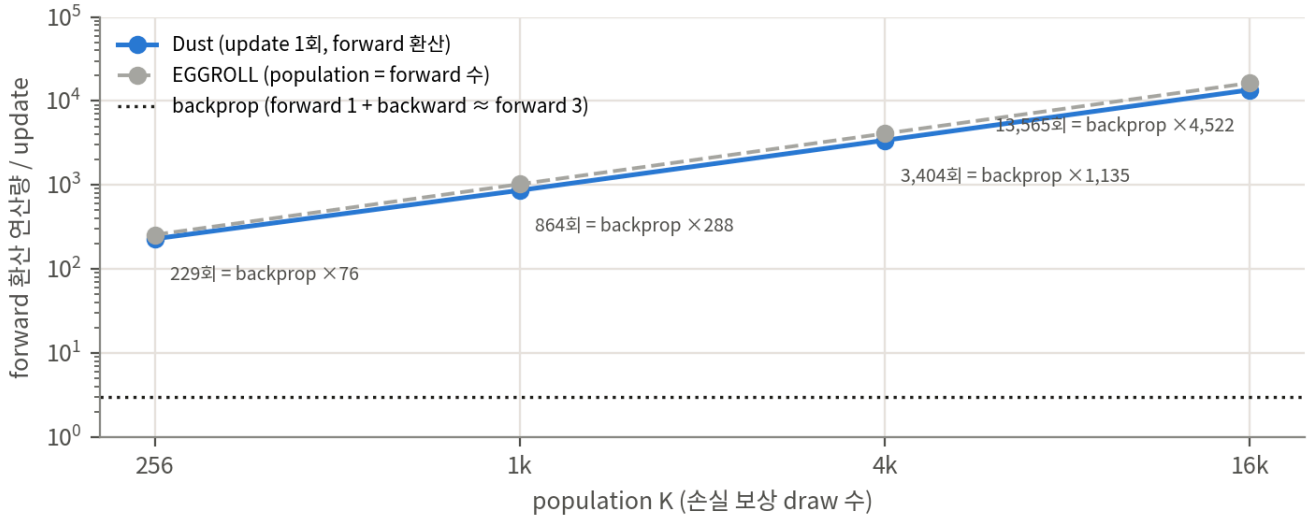


Table 1을 다시 그림(제목의 step 수는 우리 계산). 오른쪽: 20M power law 극한의 한 칸 빼기 재적합.



update 한 번의 연산량을 forward 횟수로 환산(부록 Table 3·4의 배분과 '블록 I부터 재계산' 규칙).

01. 논문 분석: Dust, 역전파 없이 Transformer 사전학습하기

- **논문:** Samip Dahal, Bishwas Mandal, Serdar Gülbahar, Akshay Vegesna. *Dust: Pretraining Transformers Without Backpropagation.* Q Labs Research, 2026년 10월.
  - 본문: https://qlabs.sh/research/dust
  - 부록: https://qlabs.sh/research/dust/appendix.html
- **원문 형태:** arXiv 판이 없는 웹 논문입니다(2026-10-29 arXiv 검색 기준).
  - 본문과 부록 HTML, 그리고 Chrome으로 인쇄한 PDF를 `paper/`에 저장했습니다.
  - 절·식·표 번호는 웹 페이지의 번호를 그대로 씁니다.
- **라이선스:** 논문 페이지에 라이선스 표기가 없습니다. 그래서 논문 그림은 개인용 리포트·PPT에만 넣고, 게시본에서는 뺍니다.
- **코드:** https://github.com/qlabs-eng/dust
  - MIT 라이선스이고, 분석 기준 커밋은 `b20f7c0`(2026-10-06)입니다.
  - README에 따르면 "추정기와 튜닝된 기본값은 유지하되 실행 최적화는 뺀 최소 구현"입니다. 1M·10M 예산, population 256-16k만 들어 있습니다.

1. Abstract

Transformer 언어모델을 **역전파 없이 forward만으로** 사전학습하는 0차(zeroth-order) 방법 Dust를 제안합니다. 두 가지가 핵심입니다.

- **가중치가 아니라 활성값을 흔듭니다:** 각 선형층의 출력에 토큰마다 독립적인 가우스 잡음을 더합니다. 이것을 node perturbation이라고 합니다.
- **보상을 토큰 단위로 줍니다:** 각 토큰의 잡음은 그 토큰의 손실 변화로 보상받습니다. 따라서 forward 한 번에 토큰 수만큼의 "가상 population"을 평가합니다.

저자들의 주장(TL;DR)은 다음과 같습니다.

1. Transformer 사전학습에서 backprop과 경쟁할 수 있는 최초의 0차 방법이고, population이 크면 여러 설정에서 backprop보다 낫습니다.
2. 가중치 공간 ES(EGGROLL)보다 10^3-10^4배 효율적입니다(외삽 기준).
3. 모델이 클수록 population 효율이 높습니다. 243M 모델이 120배 작은 모델보다 대부분의 population에서 낫습니다.
4. population이 커질수록 Dust의 gradient가 backprop의 gradient와 더 잘 정렬되고, 1B 토큰까지 정렬이 유지됩니다.

2. Method

2.1 활성값 공간의 섭동 (§2.1)

선형층 y_t = W x_t의 출력에 y_t → y_t + σ a_t (a_t ~ N(0, I))를 모든 토큰에 독립적으로 더합니다.

- 토큰 t에서 이 섭동은 그 층 가중치에 저랭크 섭동을 준 것과 같습니다. 따라서 가중치를 실제로 복사하지 않아도 됩니다.
- 시퀀스 길이 2,048이면 forward 한 번에 2,048개 "구성원"을 평가하는 셈입니다.
- 잔차 혼합 스칼라 2L개만 가중치 공간 ES로 학습합니다.

2.2 보상과 추정 (§2.2, Eq. 1-3)

- **보상:** c_s = ℓ̄ − ℓ_s입니다. ℓ̄는 같은 forward 묶음 안 draw들의 평균 손실입니다.

- **미래 토큰 credit:** $r_t = \Sigma_{s \geq t} \gamma^{s-t} c_s$ (Eq. 1). $\gamma = 0$이면 자기 토큰만 봅니다.
- **층 출력 오차 추정:** $\hat{g}_t = -(1/K\sigma) \Sigma_i r_t^{(i)} a_t^{(i)}$ (Eq. 2). K는 population, 즉 draw 수입니다.
- **가중치 gradient:** $\hat{G}_W = \Sigma_t \hat{g}_t x_t^T$ (Eq. 3). backprop과 같은 외적이고, 출력 오차를 chain rule이 아니라 population에서 얻는다는 점만 다릅니다.

2.3 간섭 줄이기와 튜닝 (§2.3, Eq. 4, 부록 E·F)

- **층 종류별 분리:** 층 종류마다 다른 forward pass에서 흔듭니다. 블록 I의 draw는 깨끗한 forward를 캐시해 두고 블록 I부터만 다시 계산합니다.
- **attention 내부(q, k, v, gate, value embedding):** 손실이 아니라 추정된 attention 출력 오차와의 정렬 $c_s = -\langle \hat{g}_s, \Delta o_s \rangle$로 보상합니다(Eq. 4). k, v, gate, ve는 미래 토큰이 읽으므로 $\gamma \approx 0.98$-0.99를 쓰고, 나머지는 $\gamma = 0$입니다.
- **LM head:** 캐시된 logit에서 256열 묶음(slab)씩만 흔들고 cross-entropy만 다시 계산합니다.
- **하이퍼파라미터 튜닝:** 층별 잡음 크기, credit decay, draw 배분을 정합니다. 배치 한 개에서 backprop gradient와의 cosine을 최대화해 후보를 고르고, 짧은 학습으로 확정합니다.
- **draw 배분(K = 16,384, 1M 기준):**
  - writer 쌍(attn·MLP 출력 투영) 7,040, MLP hidden 7,040, attention 출력 1,664, 토큰 임베딩 640
  - 앞 블록에 많이 줍니다(블록 1: 2,432 → 블록 8: 128).
  - head는 별도로 65,536 slab draw를 씁니다.

3. Experiments

3.1 설정 (§3.1, 부록 A)

- **모델:** modded-nanoGPT형 decoder 8층, 폭 512, 37.7M 파라미터.
  - RMSNorm, RoPE, squared-ReLU MLP
  - sliding window(블록 4개 중 3개는 1,024)
  - 블록 절반에 value embedding
- **데이터와 학습:** FineWeb, BPE 4,096. 배치 16,384 토큰(8 × 2,048), 1 epoch, 모멘텀 SGD, 상수 학습률.
- **평가:** 셀마다 3 시드. 검증 손실 기준으로 고른 체크포인트의 test 손실을 보고합니다.
- **비교 대상:** backprop(같은 grid로 튜닝), EGGROLL-Transformer(가중치 공간 ES, 같은 구조).

3.2 주 결과 (Table 1, Fig. 2)

예산	Dust 64	256	1k	4k	16k	Dust ∞(fit)	EGGROLL 16k	Backprop
100k	7.240	**7.165**	**7.151**	**7.164**	**7.170**	7.161	7.263	7.216
1M	6.290	6.067	**5.934**	**5.911**	**5.916**	5.892	6.708	5.959
10M	5.397	5.231	5.111	5.072	5.049	5.013	6.033	4.989
20M	5.265	5.139	4.960	4.912	4.802	**4.431**	5.865	4.633

굵은 값은 backprop보다 낮은 칸입니다. 저자 해석은 다음과 같습니다.

- 100k와 1M에서는 Dust가 backprop보다 낮습니다.
- 10M에서는 적합 극한이 backprop 바로 위입니다.
- 20M에서는 사다리가 아직 내려가는 중이고, power law 극한 4.431(95% 구간 3.89-4.58)이 backprop 4.633보다 낮습니다.

- EGGROLL은 population을 256배로 늘려도 Dust@64에 못 미칩니다.

3.3 Adam (§3.3, Fig. 3)

1M 토큰에서 세 방법을 모두 Adam으로 다시 튜닝했습니다. Dust의 극한은 5.248(구간 5.17-5.30)로, backprop보다 낮다고 보고합니다. 그림에 표시된 backprop 값은 5.361입니다. EGGROLL은 Adam으로 거의 나아지지 않습니다.

3.4 과대매개변수화 (§4.1, Fig. 4)

10M 토큰에서 2M, 7.3M, 38M, 243M 네 가지 크기를 비교했습니다. 저자 해석은 다음과 같습니다.

- 256 draw 이상에서는 2M → 7M → 38M으로 갈수록 손실이 낮아집니다. 243M은 조금 나쁩니다.
- 1k 이후 population 증가로 얻는 이득은 큰 모델이 약 30% 더 큽니다.
- 모델 크기를 "탐색 공간의 크기와 기하"로 봐야 한다고 해석합니다.

3.5 gradient 정렬 (§4.2, Fig. 5, 부록 G)

backprop으로 학습한 10M, 100M, 1B 토큰 체크포인트에서, Dust 추정치와 backprop gradient의 cosine을 population 64-128k에 걸쳐 쟀습니다.

- cos(K) = c_max/√(1 + c/K)가 층 종류마다 RMSE 0.06 이하로 맞습니다(Eq. 5).
- c_max는 head·Q·K·value emb에서 1.0이고, MLP·attn out·V에서 0.86-0.88, token emb 0.73, value gate 0.67입니다(Table 8).
- 같은 forward 수의 EGGROLL은 head를 빼면 cosine 0.05 미만입니다.

4. Limitations

4.1 저자가 밝힌 한계 (§1, §5)

- 연산 효율은 목표가 아니었습니다. backprop을 대체하려면 연산 효율이 몇 자릿수 더 좋아져야 합니다.
- 미분 불가능한 구조, 외부 프로그램이 들어간 망, 루프 구조 학습은 향후 과제입니다.
- population이 크면 backprop보다 낮아지는 메커니즘은 아직 모릅니다.

4.2 추가로 확인한 한계 (`03`, `04`에서 근거)

1. **"backprop을 넘는다"는 고정 토큰 비교이고, 연산량은 수천 배입니다.**
   - K = 16k의 update 한 번은 forward 약 13,600회, 즉 backprop step(약 forward 3회)의 약 4,500배입니다(우리 환산).
   - 같은 연산량이면 backprop은 1M 대신 수십억 토큰을 볼 수 있습니다(`03` §2).
2. **실험이 학습 극초기에 몰려 있습니다.**
   - step 수는 100k 예산에서 6, 1M에서 61, 20M에서 1,221입니다.
   - 토큰/파라미터는 최대 0.53입니다. Chinchilla 기준의 약 2.7%입니다.
   - backprop 손실도 균등 예측 손실(ln 4096 = 8.32)에서 13-44%만 내려간 구간입니다(`03` §1).
3. **20M의 "극한이 backprop보다 낮다"는 결론이 불안정합니다.**
   - 다섯 칸 중 한 칸을 빼고 다시 적합하면 극한이 2.57-4.68로 흔들립니다. 64 draw 칸을 빼면 극한이 backprop보다 높습니다.
   - 사다리 맨 위(16k) 칸만 자기 상수로 따로 튜닝되었습니다(부록 F)(`03` §3).

4. **학습 population에서의 cosine은 "가깝다"고 하기 어렵습니다.** Table 8 법칙으로 K = 16k의 cosine을 계산하면 head 0.98을 빼고 0.16-0.57입니다(`03` §4).
5. **c_max < 1인 층에는 공통점이 있습니다.**
   - c_max < 1인 층은 자기 토큰 손실만으로 보상받는 층(γ = 0이고, 출력이 뒤 토큰에도 영향을 주는 층)입니다.
   - c_max = 1인 층은 영향이 자기 토큰에만 미치거나(head, Q), 미래 credit을 받는 층(K, value emb)입니다.
   - 우리 toy는 이 해석과 맞습니다(`03` §5).
6. **과대매개변수화 결론은 약합니다.**
   - 243M은 모든 population에서 38M보다 나쁩니다. backprop에서도 243M(5.048)이 38M(5.015)보다 나쁩니다.
   - 10M 토큰은 243M에게 토큰/파라미터 0.04입니다.
   - backprop과의 차이는 64 draw에서 2M 0.525 → 243M 0.671로 커집니다(`03` §1).
7. **불확실성이 표에 없습니다.** 시드별 값과 표준편차가 보고되지 않았습니다. 100k의 Dust-backprop 차이(0.05-0.07)가 시드 변동보다 큰지 판단할 수 없습니다.

03. 재계산과 해석

재실행 방법:

```
python3 analysis/table_check.py
python3 analysis/dust_toy.py
python3 analysis/plot_figures.py
```

결과는 `table_check_results.json`과 `dust_toy_results.json`에 있고, 그래프는 `report/figures/`에 있습니다.

1. 숫자는 맞고, 실험은 학습 극초기에 있다

- **본문 수치 12개:** 부록 Table 1, Fig. 4 표, Table 8에서 다시 계산하면 모두 맞습니다(`table_check.py` V1). 맞는 수치의 예는 다음과 같습니다.
  - 100k와 1M에서 backprop보다 낮아지는 지점
  - EGGROLL@16k와 Dust@64의 차이: 0.023, 0.418, 0.636, 0.600
  - 큰 모델의 이득 비율 1.32배
- **극한 재적합:** Table 1의 ∞ 행을 다섯 칸 power law로 다시 적합하면 7.162 / 5.892 / 5.013 / 4.434로, 보고값과 같습니다.
- **파라미터 수:** 부록 A 구조로 세면 37.75M으로, 보고값 37.7M과 맞습니다.

실험이 놓인 구간은 다음과 같습니다.

예산	step 수 (16,384 토큰/step)	토큰/파라미터	backprop 손실	ln V = 8.32 대비 줄어든 비율
100k	6	0.003	7.216	13%
1M	61	0.027	5.959	28%
10M	610	0.27	4.989	40%
20M	1,221	0.53	4.633	44%

- Chinchilla식 기준(파라미터당 약 20 토큰)이면 37.7M 모델에 약 7.5억 토큰이 필요합니다. 가장 긴 실험도 그 2.7%입니다.
- "Dust가 backprop을 넘는다"는 두 칸(100k, 1M)은 **업데이트가 6번과 61번**인 구간입니다. 이 구간의 손실은 학습률, 모멘텀, 초기화, 업데이트 잡음에 크게 좌우됩니다. gradient 방향의 질을 비교하는 실험으로 읽기는 어렵습니다.
- Fig. 4의 모델 크기 비교(10M 토큰)에서 243M 모델의 토큰/파라미터는 0.04입니다. backprop에서도 243M(5.048)이 38M(5.015)보다 나쁜 구간입니다.

2. 연산량: "같은 토큰"이지 "같은 연산"이 아니다

부록 Table 3·4의 draw 배분과 "블록 l부터 다시 계산" 규칙으로 update 한 번의 연산량을 forward 횟수로 환산했습니다(`table_check.py` R2).

- **계산에 쓴 비용:**
  - 블록 하나: $12d^2 + 2 \cdot ctx \cdot d$ MAC/토큰
  - head: $d \cdot V$
  - attention 내부 draw는 그 블록의 attention만, head draw는 캐시된 logit의 256열만 다시 계산합니다.

population K	forward 환산 / update	backprop step(≈ forward 3회) 대비	같은 K의 EGGROLL 대비
256	229	76배	0.89
1k	864	288배	0.84
4k	3,404	1,135배	0.83
16k	13,565	**4,522배**	0.83

- **draw 한 번의 비용:** 평균 0.82 forward입니다. 논문의 "draw는 forward보다 조금 싸다"와 맞고, "256 이상에서 EGGROLL보다 연산이 적다"도 맞습니다.
- **같은 연산량이면:** 1M 토큰에서 K = 16k인 Dust와 같은 연산이면, backprop은 약 45억 토큰을 학습할 수 있습니다. 논문 표에서 backprop은 20M 토큰만으로 4.633에 이르고, 이는 Dust의 어떤 1M 칸(최저 5.911)보다 낮습니다. 저자들도 "연산 효율은 목표가 아니다"라고 밝혔지만, TL;DR의 "backprop을 넘는다"는 이 조건을 붙여서 읽어야 합니다.
- **10^3-10^4배 효율:** 이 수치는 **EGGROLL 대비 population(= forward 수) 효율**입니다. backprop 대비가 아닙니다.

3. 20M의 "backprop보다 낮은 극한"은 한 칸에 달려 있다

Table 1의 20M 다섯 칸에서 한 칸씩 빼고 다시 적합했습니다(`report/figures/ladders.png`).

뺀 칸	64	256	1k	4k	16k	(모두 사용)
극한	**4.679**	4.599	2.567	4.463	4.608	4.434

- **결론이 한 칸에 좌우됩니다.** 64 draw 칸을 빼면 극한이 backprop(4.633)보다 높아집니다. 1k 칸을 빼면 2.57로 발산에 가깝게 흔들립니다. 지수 α = 0.146으로 매우 작아서, 극한이 사실상 몇 개 점의 곡률로 정해집니다.
- **맨 위 칸의 조건이 다릅니다.** 20M의 16k 칸은 다른 칸과 달리 자기만의 상수로 따로 튜닝되었습니다(부록 F). 손실 보상 draw도 16,384개가 아니라 14,336개입니다(부록 E). 그래서 사다리 맨 위 칸의 하락에는 population 효과와 튜닝 효과가 섞여 있습니다.
- **해석:** 저자들도 "측정된 극한이 아니라 간격이 계속 줄어든다는 증거로 읽는다"고 썼고, 보고한 95% 구간(3.89-4.58)도 넓습니다. 따라서 "compute가 충분하면 backprop을 넘을 수 있다"는 이 적합만으로는 지지되지 않습니다.
- **EGGROLL 외삽(Table 2):** 직선 적합 두 가지는 9칸이 모두 일치합니다. power law 적합은 1M(23,000배)만 맞고, 10M과 20M은 우리 재적합에서 10,200배와 5,700배가 나와 보고값(16,000배, 8,100배)과 다릅니다. 시드 단위로 적합했거나 초기값이 달랐을 수 있습니다. 결론(10^3-10^4배)은 바뀌지 않습니다.

4. 학습 population에서의 cosine

Table 8의 법칙에 학습 population K = 16,384를 넣으면 다음과 같습니다(`report/figures/cosine_16k.png`).

층	cos@16k	c_max	c_max의 90%에 필요한 K
LM head	**0.98**	1.00	178
Attn out	0.57	0.87	5,073
MLP out	0.51	0.88	7,418
Token emb	0.49	0.73	4,574
MLP in	0.43	0.86	11,357
Attn Q	0.42	1.00	18,000

층	cos@16k	c_max	c_max의 90%에 필요한 K
Value gate	0.34	0.67	11,227
Attn V	0.26	0.88	40,763
Value emb	0.18	1.00	116,217
Attn K	**0.16**	1.00	144,633

- **cosine 수준:** 학습에 실제로 쓴 population에서 대부분 층의 cosine은 0.2-0.6입니다. "backprop gradient에 가깝다"는 K가 10만을 넘는 측정 구간(64-128k)의 이야기입니다.
- **그런데도 손실이 비슷한 이유:** 모멘텀 SGD는 여러 step에 걸쳐 잡음을 평균하므로, step마다 cosine이 0.5여도 학습이 됩니다. 다만 Table 8은 backprop으로 학습한 체크포인트에서 잰 값이라, Dust 학습 중의 cosine과 같다는 보장은 없습니다.

5. cosine 법칙의 유도와 c_max < 1의 원인 (numpy toy)

5.1 법칙의 유도

σ → 0의 선형 근사에서 한 draw의 보상은 $r_t \approx \sigma\, a_t^\top g_t$입니다. 그러면 다음이 성립합니다.

- $E[\hat{g}_t] = g_t$
- $\mathrm{Cov}(\hat{g}_t) = (\|g_t\|^2 I + g_t g_t^\top) / K$

이를 gradient $\hat{G} = \Sigma_t\, \hat{g}_t\, x_t^\top$로 모으면 다음 식이 나옵니다.

$\cos(\hat{G}, G) \approx 1 / \sqrt{(1 + c/K)}$, $c = (d_{out} + 1) \cdot \Sigma_t \|x_t\|^2 \|g_t\|^2 / \|\Sigma_t\, g_t\, x_t^\top\|^2$

- **법칙의 정체:** 논문 Eq. 5는 c_max = 1인 이 식에, 편향 c_max를 더한 형태입니다.
- **c의 의미:** "출력 차원 × 토큰별 gradient의 비정합도"입니다. 토큰마다 gradient가 서로 상쇄할수록 c가 커집니다.
- **toy 검증:** 적합한 c와 이론 c가 hidden층 308 vs 314, head 76 vs 89로 맞습니다(T1).

5.2 c_max < 1은 "자기 토큰만 보상"하는 편향이다

toy에 토큰 사이 영향(인과 평균 혼합, 세기 β)을 넣었습니다(T2).

β	cos(국소 gradient, 전체 gradient)	γ = 0 적합 c_max	γ = 1 적합 c_max	γ = 1의 c / γ = 0의 c
0	1.000	0.995	1.000	171배
1	0.936	**0.933**	1.000	177배
3	0.694	**0.692**	1.000	216배

- **γ = 0일 때:** 추정기는 전체 gradient가 아니라 "자기 토큰 손실에 대한 국소 gradient"를 추정합니다. 그래서 c_max가 정확히 cos(국소, 전체)가 됩니다.
- **γ = 1일 때:** 미래 토큰까지 보상하면 편향은 사라지지만, 보상에 T개 토큰의 잡음이 더해져 분산(c)이 170-220배 커집니다.
- **Table 8과의 대응:** Table 8의 패턴이 이 해석과 맞습니다.
  - c_max = 1인 층: LM head와 Q(출력이 자기 토큰에만 영향), K와 value emb(미래 credit을 받음)
  - c_max < 1인 층: writer, MLP, token emb(γ = 0이지만 뒤 블록의 attention을 거쳐 미래 토큰에 영향)
  - V(0.88)와 value gate(0.67)는 예외입니다. credit을 받지만 c_max < 1입니다. 이유는 확인하지 못했습니다.

- **성능 한계:** 이 편향 때문에 Dust는 population을 아무리 늘려도 MLP·writer 층에서 backprop 방향의 약 0.86-0.88까지만 다가갑니다. 논문은 이를 "다른 궤적이라 오히려 좋은 성질"이라고 해석하지만, 원인이 구조적 편향이라는 점은 밝히지 않습니다.

5.3 그 밖의 toy 결과

- **간섭(T3):** 두 층을 같은 forward에서 함께 흔들면 c_max는 그대로이고, c가 277에서 446으로 1.6배 커집니다. 즉 간섭은 편향이 아니라 분산입니다. 층 종류별로 pass를 나누는 설계가 정당합니다.
- **weight-space ES(T4):** 같은 forward 수에서 Dust가 앞섭니다(K = 64에서 cosine 0.41 vs 0.18). 다만 toy의 W1은 파라미터가 1,024개뿐이라 K = 4,096에서 ES가 0.82까지 따라옵니다. ES의 분산은 파라미터 수에 비례하므로, 37.7M 모델에서는 격차가 훨씬 큽니다.
- **묶음 중심화(T5):** n개 draw를 묶어 평균을 빼면 추정 크기가 (1 − 1/n)배가 됩니다. 측정값은 n = 2에서 0.494, 8에서 0.878, 16에서 0.942입니다. 공식 코드는 population에 따라 n = 2-16을 씁니다. 따라서 population마다 실효 학습률이 달라지고, 이는 학습률을 population별로 다시 튜닝하는 이유 중 하나일 수 있습니다. 논문 Eq. 1의 c_s = ℓ⃠ − ℓ_s는 n = 1(64 draw)에서 0이 되는데, 코드는 이때 깨끗한 손실을 baseline으로 씁니다.

6. "backprop보다 낮아진다"에 대한 다른 설명

논문은 Dust가 "고차 곡률을 암묵적으로 탐색해 평평한 영역으로 간다"는 가설을 제시합니다. 다른 설명도 가능합니다.

1. **가우스 평활화:** 유한 σ의 섭동 추정기는 평활화된 손실 E_a[ℓ(y + σa)]의 gradient를 추정합니다. 이는 잡음 주입 정규화와 같아서, 극초기 학습의 test 손실을 낮출 수 있습니다.
2. **튜닝 비대칭:** Dust는 층별 잡음 크기, credit decay, draw 배분까지 튜닝합니다. 특히 맨 위 칸은 학습으로 다시 튜닝했습니다. backprop은 모멘텀과 학습률만 튜닝합니다(부록 F).
3. **극초기 구간:** 6-61 step에서는 gradient 방향의 정확도보다 step 크기와 잡음이 손실을 더 좌우합니다.

시드별 값이 없어서 100k의 차이(0.05-0.07)가 시드 변동보다 큰지도 판단할 수 없습니다.

추가 검증 (2026-11-11)

스크립트: `analysis/extra_dust_small_lm.py` → `analysis/extra_dust_small_lm_results.json` (PyTorch CPU, 4 스레드, 실행 464초). 논문 코드를 쓰지 않고 직접 구현한 최소 Dust입니다.

추가 검증 1: 작은 문자 단위 Transformer에서 Dust, backprop, backprop + 가우스 잡음의 극초기 학습 곡선

목적. §1의 "극초기(업데이트 6-61번)에서만 backprop보다 낮다"와 "잡음 평활화로 설명할 수 있다"를 작은 모델에서 직접 봅니다. backprop + 가우스 잡음은 평활화 설명의 대조군입니다.

설정(가정).

- 모델: 문자 단위 causal Transformer, 2층, 폭 64, 헤드 4, 문맥 64, 정규화는 파라미터 없는 RMSNorm, 편향 없음. 가중치 행렬 전부를 학습합니다(임베딩과 head 포함).
- 잡음 자리: 임베딩 출력, qkv, attention 출력 투영, MLP 입력(fc1), MLP 출력(fc2), head. 잡음은 토큰마다 독립이고 σ = 0.1입니다. 모든 자리에 한 forward에서 동시에 더합니다.
- Dust: K = 32 draw. 토큰별 손실 변화 Δℓ를 미래 토큰까지 합한 보상 r_t = Σ_{s≥t} Δℓ_s(γ = 1 가정)로 ĝ_t = (1/Kσ) Σ_k r_t^(k) a_t^(k)를 만들고, Ĝ_W = Σ_t ĝ_t x_tᵀ를 가중치 gradient로 씁니다. 추정치는 Adam에 넣습니다.
- backprop과 backprop + 가우스 잡음: 같은 모델, 같은 Adam, 학습률 3e-3, 배치 8 × 64 문자, 1000 step.

- 데이터: 로컬 Python 표준 라이브러리 소스 약 200만 자(다운로드 없음), 뒤 2%를 검증. 검증 손실은 문자당 자연로그 NLL, 256 창의 평균입니다.
- 시드 2개. 장치: CPU. 패키지: torch 2.15.0, numpy 1.26.4, Python 3.12.7.

결과 (검증 NLL, nats/char, 시드 2개 평균).

업데이트 수	6	10	30	61	100	300	610	1000
backprop	3.32	3.17	2.90	2.75	2.66	2.55	2.46	2.43
backprop + 잡음	3.36	3.21	2.93	2.78	2.65	2.52	2.46	2.43
Dust (K = 32)	4.23	4.04	3.65	3.48	3.32	3.29	3.29	3.21

검증. 추정한 Dust gradient와 backprop gradient의 cosine은 초기 파라미터에서 K가 커질수록 올라갑니다. 헤드 기준으로 K = 32에서 0.13, K = 256에서 0.31, K = 1024에서 0.52입니다. 편향이 없는 추정치가 분산 때문에 방향을 잘 못 맞추는 모양이라, 부호나 축 순서의 오류는 아닌 것으로 봅니다.

결론.

- 이 작은 설정에서 Dust는 모든 업데이트 수에서 backprop보다 나쁩니다. 6번째 업데이트에서도 4.23 대 3.32로 차이가 큽니다. 논문이 100k·1M 예산에서 backprop을 이긴다고 한 결과는 여기서 재현되지 않았습니다.
- backprop + 가우스 잡음은 backprop과 거의 같습니다(차이 0.04 이하). 잡음 평활화만으로는 backprop보다 빨라지지 않습니다.
- 따라서 "극초기에만 낮아진다"는 현상을 이 toy에서 확인하지 못했습니다. 이 toy의 Dust가 나쁜 이유는 population이 작아서 추정 분산이 크기 때문입니다. 논문의 population(토큰당 수천 단위)과 batch 크기를 맞추지 않았습니다.

한계. 직접 구현한 최소 Dust입니다. 논문의 잡음 크기, 자리 선택, γ, 학습률, 정규화 세부 사항이 다를 수 있습니다. σ와 K를 튜닝하지 않았습니다. 모든 자리에 잡음을 동시에 넣었으므로 층 간 간섭이 있습니다. 시드 2개이고, 모델과 데이터가 논문 설정(37.7M, FineWeb)과 다릅니다.

04. 논문 ↔ 코드 대조와 주장 검증

- **공식 코드:** https://github.com/qlabs-eng/dust
  - 커밋 `b20f7c03eac630b6441ba6f254128c1761dc45ad` (2026-10-06), MIT 라이선스
  - 구성: `dust.py` 607줄, `model.py` 201줄, `baselines/backprop.py` 148줄
  - `code_official/` 에 clone했고, git에서는 제외합니다.
- **README의 범위:** "논문의 추정기와 튜닝된 기본값은 유지하되, 실험에 쓴 실행 최적화는 뺀 최소 구현"이라고 적혀 있습니다.
  - 1M·10M 예산과 population 256, 1k, 4k, 16k만 지원합니다.
  - 실행에는 NVIDIA GPU(bf16)가 필요합니다.

1. 논문과 코드가 일치하는 부분

항목	논문	코드	위치
draw 배분 (K = 16,384, 1M)	부록 Table 3·4	writers [38, 32, 16, 10, 6, 4, 2, 2] × 64 = [2432, ⋯, 128], hubs [6, 6, 4, 2, 2, 2, 2, 2] × 64, embedding 640, local (q, v, gate, k, ve) = (128, 1280, 384, 768, 512), head 65,536	`recipe()`
10M@16k 별도 배분과 γ	부록 Table 4, F: γ = 0.99	writers [34, 28, 14, 8, 6, 4, 2, 2] × 64, hubs [12, 10, 8, 4, ⋯] × 64, `gamma=0.99`	`DEFAULT_CONFIGS['10M']['allocation']`
K의 정의	writer·MLP·attention 출력·embedding의 손실 보상 draw 합	2·7,040 + 1,664 + 640 = 16,384 (10M@16k도 16,384)	`table_check.py` V4에서 재구성
잡음 크기	0.2, 깊은 절반 attention 출력 0.4, 내부 0.05, head 0.05, 잔차 스칼라 0.03	같음	`step()`, `local_errors`, `head_error`, `scalar_gradients`
가중치 gradient	$\hat{G} = \Sigma_t \hat{g}_t x_t^T$, embedding은 scatter-add	`reassemble()`	
attention 내부 보상	Eq. 4, Q는 자기 토큰, 나머지는 γ로 미래 토큰 합산	`local_errors`: `score` → `credit` 행렬(γ^lag, 창 안)	
잔차 스칼라	2L개는 가중치 공간 ES	양측(antithetic) ES 8 draw	`scalar_gradients`
학습 step	1M = 61, 10M = 610	같음 (배치 8 × 2,048)	`DEFAULT_CONFIGS`
학습률 그룹	부록 Table 6: 토큰 임베딩 1000, value emb 0.3, 잔차 0.03	같음. backprop은 임베딩 1000, 나머지 base	`make_optimizer` (두 파일)
backprop 기준선의 공정성	같은 모델·데이터·평가	같은 `make_model` (출력 투영 초기화 포함), 같은 데이터 순서(seed + 1), 같은 평가	`baselines/backprop.py`

2. 논문과 코드가 다른 부분, 또는 논문에 없는 세부 (D1-D8)

#	항목	논문	코드	영향
D1	보상 baseline	c_s = ℓ⊠ – ℓ_s (같은 묶음 draw 평균과의 차, Eq. 1 앞)	깨끗한 손실 – 섭동 손실을 구한 뒤, 묶음(n ≥ 2) 안 평균을 뺌. n = 1이면 중심화 없음	n = 1이면 논문 식은 0이 되므로 코드가 맞는 구현. 묶음 중심화는 추정 크기를 (1 – 1/n)배로 줄임(`03` §5.3)
D2	head 보상의 baseline	명시 없음	**draw 묶음이 아니라 배치의 8개 시퀀스에서 같은 위치의 평균**을 뺌(`reward.mean(1)`)	다른 시퀀스의 손실을 baseline으로 쓰는 셈. 편향은 작지만 논문 서술과 다름
D3	gate의 섭동	일반적인 한쪽 섭동으로 서술	gate만 양측 차분(z + σa, z – σa), 나눗셈 2σ	분산 감소. 논문에 없음
D4	writer 쌍	"층 종류마다 다른 pass"(§2.3)	attn.c_proj와 mlp.c_proj를 **같은 pass에서 함께** 흔듦(부록 Table 3의 "Writer pair")	같은 블록의 두 출력 사이에 간섭이 생김. toy에서 함께 흔들면 c가 1.6배(`03` §5.3)
D5	지원 범위	100k, 1M, 10M, 20M × 64-16k, Adam, 모델 크기 4종	1M·10M × 256-16k, SGD만. 64 draw(n = 1), 100k, 20M, Adam, 모델 크기 설정 없음	논문 핵심 수치 중 20M 극한(4.431), Adam 결과, 과대매개변수화는 공개 코드로 재현할 수 없음
D6	EGGROLL-Transformer	같은 구조로 구현, 자체 grid 튜닝	저장소에 없음	Table 1의 EGGROLL 행과 Table 2 외삽을 확인할 수 없음
D7	시드·튜닝 절차	seed 42로 grid, 상위 후보에 43·44 (부록 F)	`--seed` 인자 하나. grid 스크립트 없음	3 시드 평균과 선택 절차를 재현하려면 직접 작성해야 함
D8	실행 최적화	블록 l부터 재계산, GPU 분할	최소 구현이라 블록 l 섭동도 forward 전체를 다시 계산(`self.losses(x, y)`)	결과는 같고 시간만 더 듦. 연산량 환산(`03` §2)은 논문 방식(블록 l부터) 기준

3. 수치 정합성 (V1-V4)

#	확인	결과
V1	본문 수치 12개(§3.2, §4.1, §4.2) ↔ 부록 표	모두 일치
V2	Table 1의 ∞ 행을 power law로 재적합	4칸 모두 ±0.003 이내로 일치. 다만 20M은 한 칸 빼기에 매우 민감(`03` §3)
V3	Table 2(EGGROLL 외삽)	직선 적합 6칸 일치, power law는 1M만 일치(10M 10,200배 vs 16,000배, 20M 5,700배 vs 8,100배)
V4	코드의 draw 배분 ↔ 부록 Table 3·4	8개 항목 모두 일치, K = 16,384

4. 주장별 판정

판정: ✅ 지지됨 · ⚠️ 부분적으로 지지되거나 해석에 주의 · ❌ 근거 부족

#	주장 (위치)	판정	근거
C1	Transformer 사전학습에서 backprop과 경쟁할 수 있는 최초의 0차 방법 (TL;DR, §1)	⚠️	같은 토큰 수에서 손실이 비슷한 것은 맞음. 다만 실험은 최대 1,221 step, 토큰/파라미터 0.53 이하이고, 연산은 backprop의 수천 배(`03` §1-2)
C2	population이 크면 여러 설정에서 backprop보다 낮다 (§3.2)	⚠️	100k(6 step), 1M(61 step)에서만 측정값으로 낮음. 차이 0.04-0.07이고 시드별 값 없음. 10M·20M은 측정값이 모두 backprop보다 높음

#	주장 (위치)	판정	근거
C3	20M 극한 4.431 < backprop 4.633 (§3.2, Fig. 2)	❌	한 칸 빼고 재적합하면 2.57-4.68. 64 칸을 빼면 backprop보다 높음. 맨 위 칸은 별도 튜닝(`03` §3). 저자도 "측정된 극한이 아님"이라고 씀
C4	가중치 공간 ES보다 10^3-10^4배 효율 (TL;DR, 부록 D)	✅ (population 기준)	외삽 산술은 대체로 재현. "효율"은 같은 손실에 필요한 population 배수이고, draw당 연산은 0.82 forward라 연산 기준으로도 비슷함
C5	Adam에서도 backprop보다 극한이 낮다 (§3.3)	⚠️	그림만 있고 표가 없음(backprop 5.361, Dust 극한 5.248). 공개 코드에 Adam 설정 없음(D5)
C6	큰 모델이 population 효율이 더 높다 (§4.1)	⚠️	2M → 38M은 맞음. 243M은 모든 population에서 38M보다 나쁘고, backprop과의 차이도 64 draw에서 0.525 → 0.671로 커짐. 243M에게 10M 토큰은 토큰/파라미터 0.04
C7	population이 커질수록 gradient가 backprop에 정렬, 1B 토큰까지 유지 (§4.2)	✅	법칙 적합 RMSE 0.0032-0.0367 확인. 다만 학습 population(16k)의 cosine은 head를 빼면 0.16-0.57(`03` §4)
C8	근사가 backprop과 정확히 같지 않은 것이 오히려 좋은 성질이다 (§4.2)	❌ (해석)	c_max < 1의 주요 원인은 자기 토큰만 보상하는 구조적 편향으로 설명됨(toy에서 c_max = cos(국소, 전체 gradient)). 더 좋은 궤적이라는 근거는 C2의 극초기 결과뿐(`03` §5-6)
C9	활성값 섭동이 가중치 섭동보다 population을 싸게 만든다 (§2.1)	✅	toy에서 같은 forward 수로 cosine 0.41 vs 0.18(K = 64). 이론상 분산이 파라미터 수가 아니라 d_out × 토큰 비정합도에 비례

5. 구현 가능성

5.1 공식 코드로 재현하는 경우

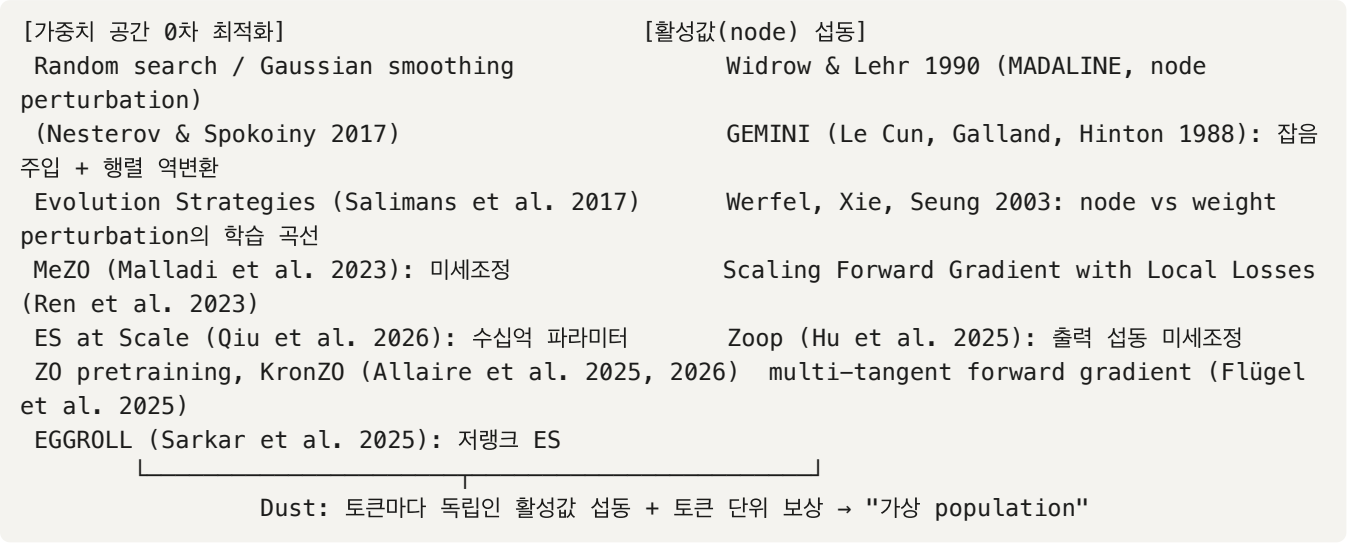
- **요구 사항:** 공식 코드는 CUDA GPU만 받습니다(`parser.error('Training requires a CUDA GPU.')`).
- **1M × 256 단일 GPU:** 61 step × update당 약 229 forward이므로 forward 약 1.4만 번입니다. 최소 구현에서는 블록 l부터가 아니라 전체 forward를 다시 계산하므로 더 걸립니다. GPU 한 장에서 시간 단위로 예상합니다(우리 추정).
- **10M × 16k:** 610 step × 13,565 forward ≈ 830만 forward이고, 8 GPU를 씁니다.
- **이 기기:** GPU 재현이라 하지 않습니다.

5.2 CPU로 가능한 것

- **numpy toy(`dust_toy.py`):** 추정기 식, cosine 법칙, γ의 편향-분산 교환을 확인했습니다(이 문서 §4의 C8, C9 근거).
- **아주 작은 Transformer:** 2층, 폭 64, 문맥 64 정도라면 CPU PyTorch로 Dust vs backprop을 몇백 step 돌릴 수 있습니다. torch 설치가 필요합니다.

02. 선행연구: Dust는 어디에서 왔는가

1. 계보



- **가중치 공간 ES:** population의 구성원마다 가중치 사본과 forward가 필요합니다. 그래서 population을 늘리는 비용이 큽니다. EGGROLL은 저랭크 섭동으로 사본 비용을 줄였지만, 구성원 하나가 배치의 한 시퀀스입니다.
- **node perturbation:** 출력 차원(d_out)이 가중치 차원(d_out × d_in)보다 작다는 차원 논리가 근거입니다. Transformer에서는 토큰 T개 × d_out의 잡음이 가중치 수보다 많아서, 이 논리가 그대로 성립하지 않습니다(§2.1).
- **Dust의 관점 전환:** 토큰 축을 배치 축과 직교하는 **새 population 축**으로 씁니다. 토큰마다 보상이 따로 있으므로, forward 한 번에 T개 구성원을 평가합니다.

2. 구성 요소별 출처

구성 요소	가져온 곳	Dust에서 바뀐 점
출력 섭동 → 출력 오차 → 입력과의 외적	node perturbation (Widrow & Lehr 1990; Werfel et al. 2003), GEMINI (1988)	토큰별 보상, 층 종류별 분리 pass, 캐시된 forward에서 블록 I부터만 재계산
미래 토큰 credit (γ)	강화학습의 할인 return과 같은 형태	attention 내부(k, v, gate, ve)에만 γ ≈ 0.98-0.99
attention 내부의 국소 목표	Ren et al. 2023의 local loss와 같은 발상	손실 대신 추정된 attention 출력 오차와의 내적으로 보상(Eq. 4)
보상 중심화	ES의 baseline / fitness shaping	같은 forward 묶음(n = 1-16 draw)의 평균을 뺌. 크기가 (1 − 1/n)배로 줄어듦(`03` §5)
모델	modded-nanoGPT 계열(RMSNorm, RoPE, squared-ReLU, value embedding, sliding window)	출력 투영을 0이 아닌 값으로 초기화(backprop 기준선도 같음)
비교 ES	EGGROLL (Sarkar et al. 2025)	같은 Transformer 구조로 다시 구현(EGGROLL-Transformer)

3. 가까운 대안과의 차이

방법	무엇을 흔드나	보상 단위	forward 한 번의 정보	Dust와의 차이
EGGROLL	가중치(저랭크)	시퀀스 손실	구성원 1개/시퀀스	Dust는 토큰마다 구성원
MeZO	가중치(전체)	배치 손실	방향 1개	미세조정용. 사전학습에는 약함
Forward gradient (Ren et al. 2023)	활성값 + forward-mode AD	국소 손실	방향 도함수(정확)	Dust는 AD 없이 유한 차분
Zoop	출력	토큰 손실	출력 오차 추정	미세조정, 국소 미분 사용
KronZO	가중치(Kronecker 구조)	배치 손실		사전학습. 메모리 절감 중심
예측 부호화 PC-ALM (Seely & Gould 2026)	없음	국소 동역학		국소 미분 사용, 이미지 분류

4. 논문이 말하지 않은 관련 연구

- **REINFORCE / likelihood-ratio gradient:** Eq. 2는 가우스 정책의 score function 추정기와 같은 꼴입니다. 분산을 줄이는 기법인 baseline, control variate, antithetic sampling은 그 문헌에 많이 있습니다. Dust는 묶음 평균만 baseline으로 씁니다.
- **SPSA, randomized smoothing:** 유한 σ에서 추정하는 것은 실제 손실이 아니라 가우스로 평활화한 손실의 gradient입니다. 논문이 "backprop보다 낮아지는" 이유로 추측하는 "평평한 영역으로 끌리는 효과"는 이 평활화로 설명될 가능성이 있습니다(`03` §6). 논문은 이 연결을 언급하지 않습니다.
- **noise injection 정규화(Gaussian noise, dropout):** 학습 극초기(6-61 step)에 backprop보다 낮은 손실이 나오는 현상은, gradient의 질보다 잡음이 섞인 업데이트와 튜닝 차이의 영향일 수 있습니다.

5. 이 저장소의 다른 논문과의 관계

논문	관계
DDPG (`ddpg/`)	같은 "가우스 탐색 잡음 + 보상" 구조입니다. DDPG의 OU 잡음 분석(상관 시간, 정상 표준편차)이 Dust의 σ 선택과 대응합니다. Dust의 Eq. 2는 결정적 정책 gradient가 아니라 확률적 정책의 score function 추정에 해당합니다.
GQKAE (`gqkae/`)	미분하기 어려운 생성기(양자 회로)를 GRPO식 보상으로 학습합니다. Dust가 노리는 "미분 불가능한 구조 학습"의 한 사례입니다.
PINN 연구 전반	PINN의 잔차 손실은 자동미분이 전제입니다. 시뮬레이터나 접촉이 섞여 미분할 수 없는 물리 모듈이 루프에 들어가면, Dust식 활성값 섭동이 대안이 될 수 있습니다. 다만 연산량 문제가 있습니다(`03` §2).
GPT-as-Tactile-Policy, AW-0	둘 다 외부 프로그램(LLM 도구 호출, Code-as-Policies)이 정책 루프에 들어갑니다. 저자들이 Dust로 학습하고 싶다고 한 "외부 프로그램이 루프에 있는 망"의 예입니다.